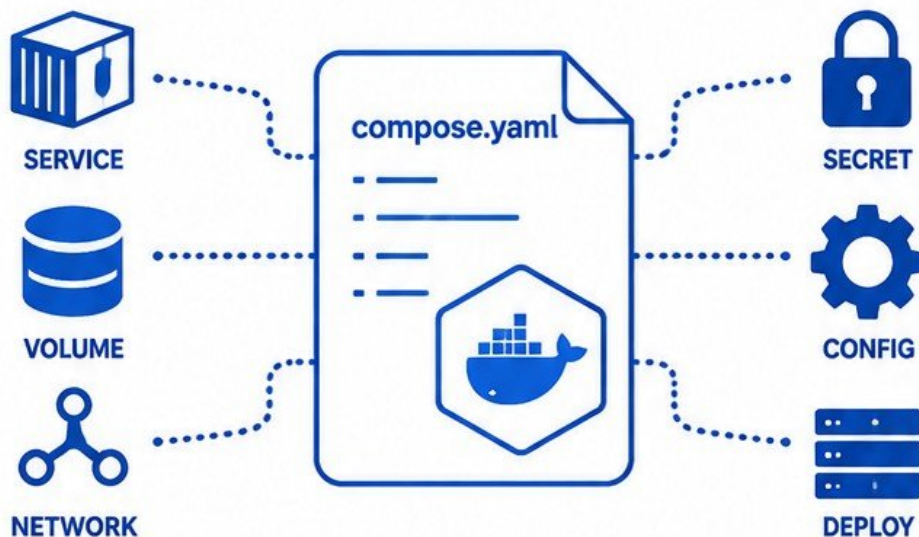


VERIQTA

Docker Compose

PRODUCTION HANDBOOK

BUILD • CONFIGURE • DEPLOY • OPERATE



PRODUCTION
READY



OPERATIONAL
EXCELLENCE



SECURE BY
DESIGN



RESILIENT &
SCALABLE



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

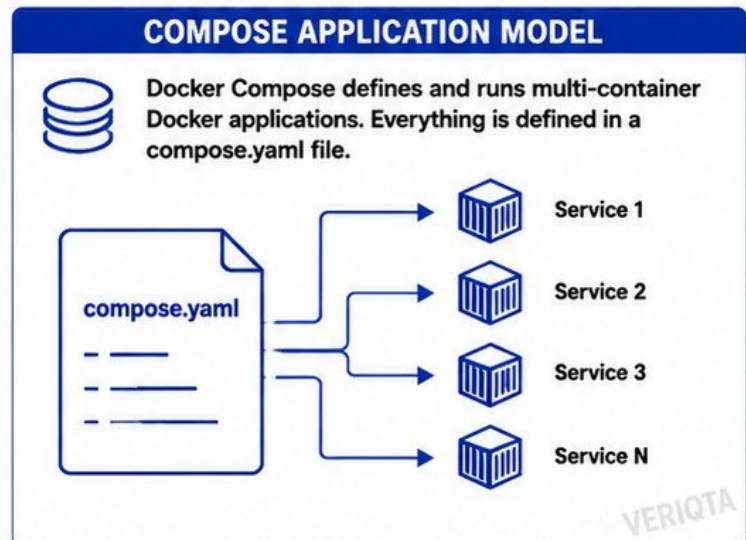
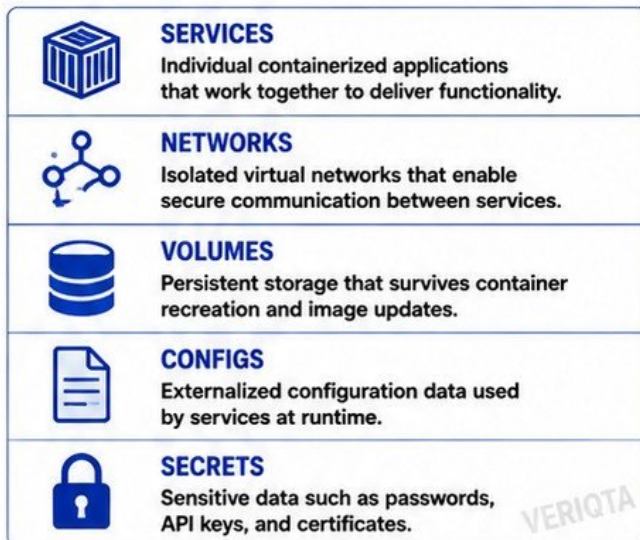


/VERIQTA

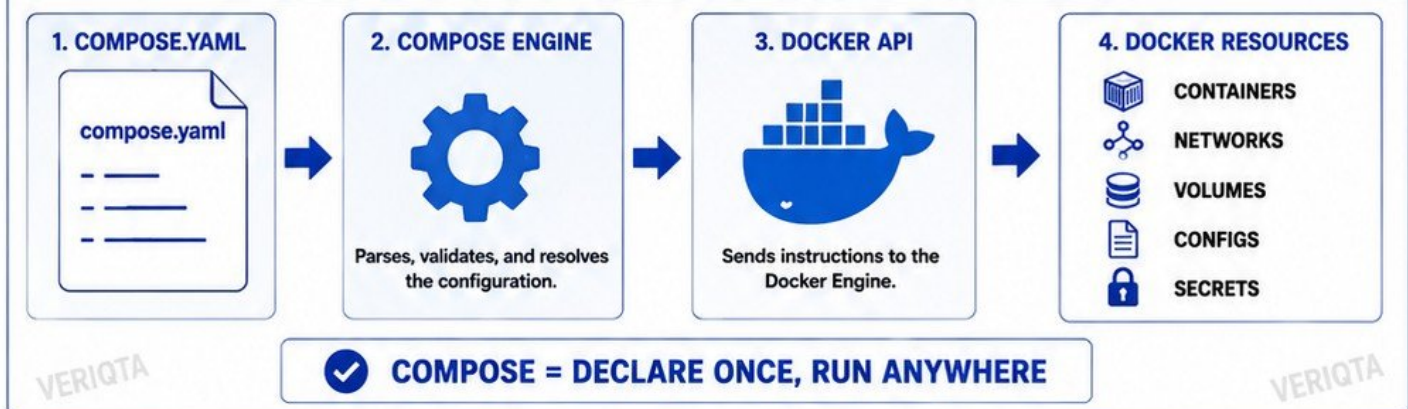
DOCKER COMPOSE ARCHITECTURE



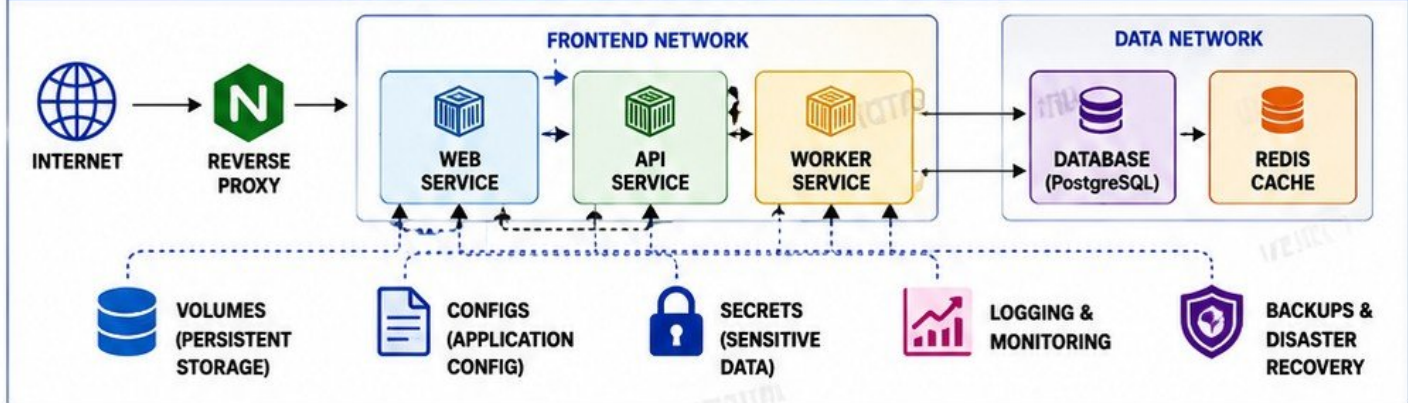
THE BUILDING BLOCKS OF MODERN CONTAINERIZED APPLICATIONS



HOW COMPOSE TRANSLATES YAML INTO DOCKER RESOURCES



PRODUCTION ARCHITECTURE DIAGRAM (EXAMPLE)



COMPOSE YAML STRUCTURE

UNDERSTAND EVERY KEY SECTION IN YOUR COMPOSE FILE



COMPOSE.YAML

- The entry file for your application.
- Defines all services and their relationships.



SERVICE DEFINITIONS

- Each service is a container.
- Defined under the services section.
- Has its own configuration.



IMAGE VERSUS BUILD

- **image:** Uses a pre-built image from a registry.
- **build:** Builds an image from a Dockerfile.
- Use image in production, build for custom apps.



PORTS

- Maps host ports to container ports.
- Format: "host_port:container_port".
- Publish only what is required.



ENVIRONMENT

- Pass configuration to containers.
- Use environment or env_file.
- Never hardcode secrets.



VOLUMES

- Named volumes or bind mounts.
- Persist data and share files.
- Protects data from container recreation.



NETWORKS

- Enable communication between services.
- Services discover each other by name.
- Isolate traffic using custom networks.



BEST PRACTICES

- Use meaningful service names.
- Keep configs in env files and secrets.
- Use named volumes for persistence.
- Don't expose unnecessary ports.
- Use healthchecks and restart policies.
- Use separate networks for isolation.

PRODUCTION-READY COMPOSE.YAML EXAMPLE

```
version: "3.9"
services:
  web:
    image: nginx:1.24-alpine           # Use official image
    container_name: web
    ports:
      - "80:80"                       # Host:Container port
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro # Read-only config
    depends_on:
      - api
    restart: unless-stopped
    networks:
      - frontend

  api:
    build:
      context: ./api
      dockerfile: Dockerfile
    container_name: api
    ports:
      - "5000:5000"                   # API exposed for testing
    env_file:
      - ./envs/api.env                # Environment variables
    environment:
      - NODE_ENV=production
    volumes:
      - ./api/uploads:/app/uploads    # Persistent uploads
    depends_on:
      db:
        condition: service_healthy    # Wait for DB ready
    restart: unless-stopped
    networks:
      - frontend
      - backend

  db:
    image: postgres:15-alpine         # Official DB image
    container_name: db
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes:
      - db_data:/var/lib/postgresql/data # Persistent DB data
    secrets:
      - db_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped
    networks:
      - backend

volumes:
  db_data:
    driver: local
secrets:
  db_password:
    file: ./secrets/db_password.txt
networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge
# Internal network for DB/API
```

VERSION
Compose file format version.

WEB SERVICE
Nginx reverse proxy for the application.

IMAGE VS BUILD
Web uses an image. API is built from source.

PORTS
Only required ports are published.

ENVIRONMENT
Load variables from env file and set runtime environment.

VOLUMES
Persist uploads and database data.

NETWORKS
Frontend for public traffic. Backend for internal communication.

SECRETS
Sensitive data loaded from file-based secret.

RESOURCES
Named volumes, secrets, and custom networks declared here.



COMPOSE YAML STRUCTURE

UNDERSTAND EVERY KEY SECTION IN YOUR COMPOSE FILE



COMPOSE.YAML

compose.yaml

- The entry file for your application.
- Defines all services and how they work together.



SERVICE DEFINITIONS

- Each service is a container.
- Defined under the services section.
- Has its own configuration.



IMAGE VERSUS BUILD

- **image:** Uses a pre-built image from a registry.
- **build:** Builds an image from a Dockerfile.
- Use image in production, build for custom apps.



PORTS

- Maps host ports to container ports.
- Format: "host_port:container_port".
- Publish only what is required.



ENVIRONMENT

- Pass configuration to containers.
- Use environment or env_file.
- Never hardcode secrets.



VOLUMES

- Named volumes or bind mounts.
- Persist data and share files.
- Protects data from container recreation.



NETWORKS

- Enable communication between services.
- Services discover each other by name.
- Use custom networks for isolation.



BEST PRACTICES

- Use meaningful service names.
- Keep configs in env files and secrets.
- Use named volumes for persistence.
- Don't expose unnecessary ports.
- Use healthchecks and restart policies.
- Use separate networks for isolation.



QUICK TIP

Compose turns this YAML into Docker resources: containers, networks, volumes, configs, and secrets.

PRODUCTION-READY COMPOSE.YAML EXAMPLE

```

1 version: "3.9"
2 services:
3   web:
4     image: nginx:1.24-alpine # Use official image
5     container_name: web
6     ports:
7       - "80:80" # Host:Container
8     volumes:
9       - ./nginx/conf.d:/etc/nginx/conf.d:ro # Read-only config
10    depends_on:
11      - api
12    restart: unless-stopped
13    networks:
14      - frontend
15  api:
16    build:
17      context: ./api # Build from Dockerfile
18      dockerfile: Dockerfile
19      target: production
20    container_name: api
21    ports:
22      - "5000:5000" # Exposed for testing
23    env_file:
24      - ./envs/api.env # Environment variables
25    environment:
26      - NODE_ENV=production
27    volumes:
28      - ./api/uploads:/app/uploads # Persistent uploads
29    depends_on:
30      db:
31        condition: service_healthy # Wait for DB ready
32        restart: unless-stopped
33    networks:
34      - frontend
35      - backend
36  db:
37    image: postgres:15-alpine # Official DB image
38    container_name: db
39    environment:
40      POSTGRES_DB: appdb
41      POSTGRES_USER: appuser
42      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
43    volumes:
44      - db_data:/var/lib/postgresql/data # Persistent DB data
45    secrets:
46      - db_password
47    healthcheck:
48      test: ["CMD-SHELL", "pg_isready -U appuser -d appdb"]
49      interval: 10s
50      timeout: 5s
51      retries: 5
52    restart: unless-stopped
53    networks:
54      - backend
55  volumes:
56    db_data:
57      driver: local
58  secrets:
59    db_password:
60      file: ./secrets/db_password.txt
61  networks:
62    frontend: # Public-facing network
63    backend: # Internal network for DB/API
64
```



CONTAINER



NETWORK



VOLUME



CONFIG



SECRET

MULTI-CONTAINER APPLICATION DESIGN



BUILD MODULAR, SCALABLE, AND MAINTAINABLE APPLICATIONS

APPLICATION DECOMPOSITION

Break your application into independent services that each have a single responsibility.



FRONTEND

Handles user interface and user interactions.



API SERVICE

Exposes business logic and REST/GraphQL APIs.



DATABASE

Stores and manages persistent data.



CACHE

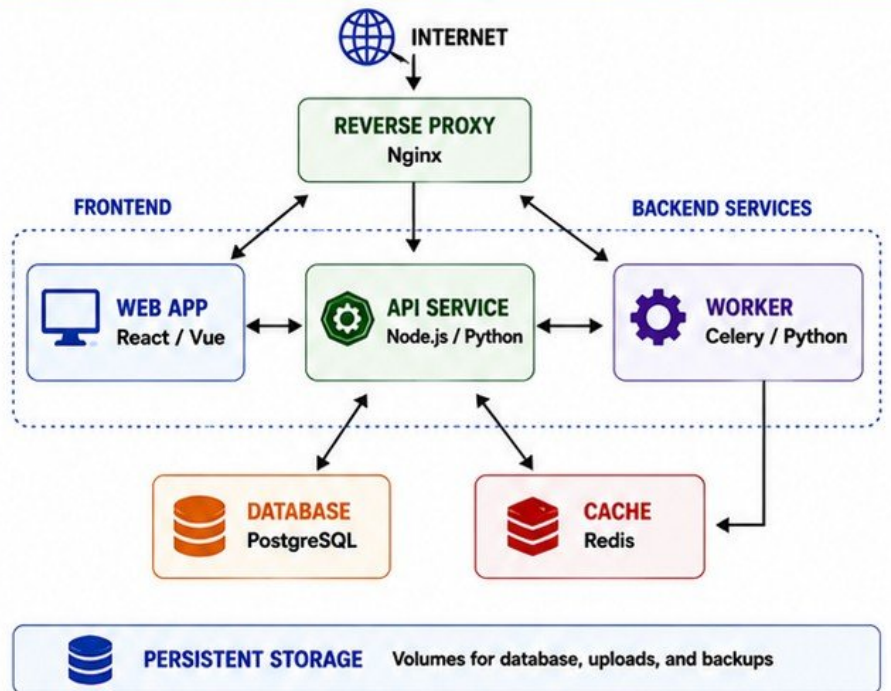
Speeds up access to frequently used data.



WORKER

Processes background jobs and async tasks.

REALISTIC MULTI-SERVICE STACK (EXAMPLE)



SERVICE BOUNDARIES

- ✓ Each service has a clear responsibility.
- ✓ Services own their data.
- ✓ Minimize shared dependencies.
- ✓ Communicate through well-defined interfaces (HTTP, gRPC, queues).
- ✓ Loose coupling enables independent development and deployment.

INTERNAL COMMUNICATION



SYNCHRONOUS

Use HTTP/HTTPS or gRPC for request/response.



ASYNCHRONOUS

Use message queues (e.g., RabbitMQ, Redis, SQS) for background processing.



SERVICE DISCOVERY

Services communicate using service names via Docker DNS.

DEPENDENCY RELATIONSHIPS

- 1 Frontend depends on API.
- 2 API depends on Database and Cache.
- 3 Worker depends on API, Database and Cache.
- 4 Database and Cache are independent infrastructure services.
- 5 Reverse Proxy is the single entry point from the Internet.

DESIGNING A REALISTIC MULTI-SERVICE STACK



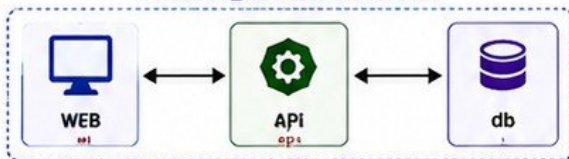
SERVICE NETWORKING AND DNS

CONNECT SERVICES SECURELY AND RELIABLY

1. DEFAULT COMPOSE NETWORK

If you don't define any networks, Docker Compose creates a default network for your project.

PROJECT_DEFAULT NETWORK



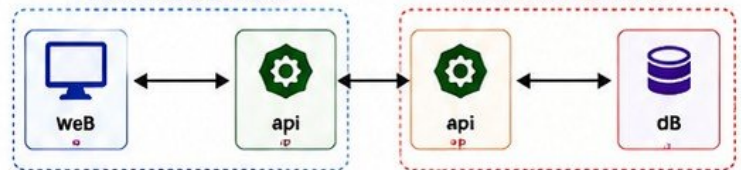
- ✓ All services can communicate with each other.
- ✓ Isolated from other Compose projects.
- ✓ Network name: `<project>_default`

2. CUSTOM NETWORKS

Define custom networks to separate traffic by purpose.

FRONTEND NETWORK

BACKEND NETWORK



- ✓ Better isolation and security.
- ✓ Control which services can communicate.
- ✓ Attach a service to multiple networks if needed.

3. SERVICE DISCOVERY

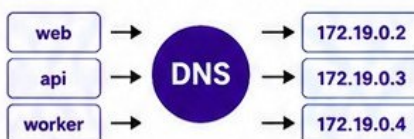
Services discover each other automatically using service names.



- ✓ Use the service name as the hostname.
- ✓ Example: web can reach api at `http://api:5000`
- ✓ No need for IP addresses.

4. CONTAINER DNS

Docker's internal DNS resolves service names to container IP addresses.



- ✓ Built-in DNS server at `127.0.0.11`
- ✓ Resolves service names in the network.
- ✓ Updates automatically when containers change.

5. SERVICE NAMES

Always use service names for communication, not localhost or container IPs.

DO THIS ✓

NOT THIS ✗

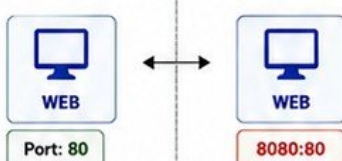
<code>http://api:5000</code> ✓	<code>http://localhost:5000</code> ✗
<code>http://db:5432</code> ✓	<code>http://172.19.0.3:5432</code> ✗

- ✓ Service names are stable.
- ✓ IP addresses can change.
- ✓ Works across restarts and rescheduling.

6. INTERNAL VS PUBLISHED PORTS

INTERNAL PORTS
(Inside the network)

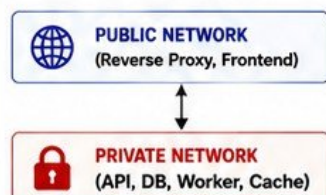
PUBLISHED PORTS
(Access from host)



- ✓ Internal: Service-to-service communication (e.g., 80).
- ✓ Published: Expose to host/Internet (e.g., 8080:80).
- ✓ Expose only what is necessary.

7. NETWORK ISOLATION

Use networks to isolate environments and reduce attack surface.



- ✓ Keep databases off public networks.
- ✓ Limit exposure of sensitive services.
- ✓ Apply least-privilege connectivity.

8. DEBUGGING CONNECTIVITY FAILURES

Tools and steps to troubleshoot network issues.

- GET CONTAINER IP
`docker inspect <container>`
- CHECK CONNECTIVITY
`docker exec -it <container> sh`
`ping <service_name>`
- CHECK PORTS
`ss -tulnp inside container`
- CHECK NETWORKS
`docker network ls`
`docker network inspect <network>`

- ✓ Validate service names.
- ✓ Check logs for errors.
- ✓ Verify exposed ports.
- ✓ Confirm network attachments.
- ✓ Firewall or security group rules.

QUICK REFERENCE



DEFAULT NETWORK
`<project>_default`
All services connected.



SERVICE DISCOVERY
Use service name as hostname.
No IP needed.



CONTAINER DNS
Built-in DNS at `127.0.0.11`
Resolves names.



ISOLATION
Use custom networks.
Limit exposure.



PORTS
Internal for services.
Published for external access.

PERSISTENT STORAGE AND VOLUMES

KEEP YOUR DATA SAFE, PERSISTENT, AND RELIABLE

1. NAMED VOLUMES



- Managed by Docker.
- Stored in Docker host filesystem.
- Portable across containers.
- Ideal for databases and persistent application data.

EXAMPLE (docker-compose.yml)

```
services:
  db:
    image: postgres:15
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
    driver: local
```

2. BIND MOUNTS

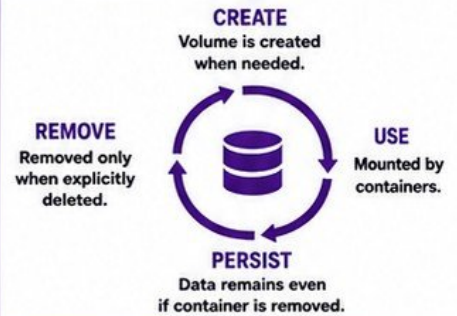


- Map a host directory or file into the container.
- Useful for development, configs, and logs.
- Host path depends on the machine.

EXAMPLE (docker-compose.yml)

```
services:
  web:
    image: nginx:alpine
    volumes:
      - ./html:/usr/share/nginx/html
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
```

3. VOLUME LIFECYCLE



COMMANDS

- `docker volume ls` # List volumes
- `docker volume inspect <name>` # Inspect
- `docker volume rm <name>` # Remove
- `docker volume prune` # Clean unused

4. DATABASE PERSISTENCE



- Always store database data in a persistent volume.
- Ensures data survives container restarts, updates, or failures.
- Never store DB data in the container layer.

EXAMPLE (PostgreSQL)

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: strongpass
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

5. READ-ONLY MOUNTS



- Mount files or directories as read-only.
- Prevents accidental modifications.
- Improve security for configs and static data.

EXAMPLE (Read-only config)

```
services:
  app:
    image: myapp:latest
    volumes:
      - ./config/app.conf:/app/config/app.conf:ro
```

6. BACKUP CONSIDERATIONS



- Back up volume data regularly.
- Use filesystem-level or application-level backups.
- Store backups outside the Docker host.
- Test restore procedures.

BACKUP METHODS

- Stop container for consistent backup
- Use `docker run --rm -v <volume>:/data alpine tar czf /backup/backup.tar.gz /data`
- Use database native backup tools
- Automate backups and retention

7. PERMISSION PROBLEMS



- Host and container users may differ.
- Result: Permission denied errors.
- Solutions:
 - Use correct user IDs
 - Adjust permissions
 - Use named volumes for database data

EXAMPLE (Fix permissions)

```
services:
  app:
    image: myapp
    user: "1000:1000"
    volumes:
      - app_data:/data
volumes:
  app_data:
```

8. PREVENTING ACCIDENTAL DATA LOSS

- ✓ Never use anonymous volumes in production.
- ✓ Always name your volumes.
- ✓ Avoid binding critical data to host paths.
- ✓ Use backups and replication.
- ✓ Document volume usage and dependencies.

BEST PRACTICES

- ✓ Use named volumes for persistent data
- ✓ Keep data outside container layer
- ✓ Backup regularly
- ✓ Validate restore process
- ✓ Use read-only mounts where possible
- ✓ Monitor disk usage
- ✓ Restrict host access to volume paths

QUICK COMPARISON

FEATURE	NAMED VOLUME	BIND MOUNT
MANAGED BY	DOCKER	HOST
PORTABILITY	HIGH	LOW
PERFORMANCE	HIGH	DEPENDS
BEST FOR	DATA, DB, SHARED	DEV, CONFIG, LOGS
LIFECYCLE	DOCKER MANAGED	HOST MANAGED
BACKUP	EASY	MANUAL
SECURITY	BETTER ISOLATION	EXPOSES HOST PATHS



Use the right type of volume for the job. Protect your data, plan your backups, and design for long-term reliability.





ENVIRONMENT VARIABLES AND CONFIGURATION

MANAGE CONFIGURATION CLEANLY AND SECURELY

1. ENVIRONMENT



- Define variables directly inside the compose file.
- Best for simple key-value pairs.
- Visible in the file.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    environment:
      - NODE_ENV=production
      - LOG_LEVEL=info
      - API_URL=http://api:5000
```

2. ENV_FILE



- Load variables from an external file.
- Keeps compose file clean and reusable.
- Good for many variables.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    env_file:
      - ./env
      - ./secrets.env
```

3. .ENV FILE



- The .env file in the project root.
- Used for variable interpolation.
- Not automatically passed to containers.

EXAMPLE (.env)

```
APP_NAME=MyApp
APP_ENV=production
DB_USER=appuser
DB_PASSWORD=supersecret
DB_HOST=db
DB_PORT=5432
```

4. VARIABLE INTERPOLATION



- Use variables in compose files.
- Syntax: \${VAR}.
- Supports default and required forms.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:${APP_TAG:-latest}
    ports:
      - "${APP_PORT:-8080}:80"
    environment:
      DB_HOST: ${DB_HOST}
      DB_PORT: ${DB_PORT:-5432}
```

5. REQUIRED VARIABLES



- Fail fast if a variable is missing.
- Use \${VAR?error} syntax.
- Prevents silent misconfiguration.

EXAMPLE

```
services:
  app:
    environment:
      DATABASE_URL: ${DATABASE_URL?
      DATABASE_URL is required}
```

6. DEFAULT VALUES



- Provide fallback values when not set.
- Use \${VAR:-default}.
- Keeps environments flexible.

EXAMPLE

```
services:
  app:
    environment:
      LOG_LEVEL: ${LOG_LEVEL:-info}
      TIMEOUT: ${TIMEOUT:-30}
      RETRIES: ${RETRIES:-3}
```

7. ENVIRONMENT-SPECIFIC CONFIGURATION



- Use multiple env files for different stages.
- Example: .env.dev, .env.staging, .env.prod
- Combine with Compose override files.

EXAMPLE

```
# Development
docker compose --env-file .env.dev up

# Staging
docker compose --env-file .env.staging
-f docker-compose.yml -f docker-compose
.staging.yml up

# Production
docker compose --env-file .env.prod up -d
```

8. AVOIDING CONFIGURATION DRIFT



- Centralize configuration.
- Use version control for env files (without secrets).
- Validate configuration on startup.
- Document all variables.
- Use templates and defaults.

BEST PRACTICES

- ✓ Use a single source of truth.
- ✓ Keep sensitive values in secrets.
- ✓ Validate required variables.
- ✓ Use consistent naming.
- ✓ Review changes in code reviews.
- ✓ Automate environment checks.

HOW COMPOSE PROCESSES VARIABLES



1. LOAD .ENV FILE
Compose reads the .env file in the project root.



2. INTERPOLATE VARIABLES
Replace \${VAR} in the compose file with values.



3. LOAD ENV_FILES (IF ANY)
Load variables from env_file entries for the service.



4. PASS TO CONTAINER ENVIRONMENT
Final environment is passed to the container at runtime.

PRECEDENCE (HIGHEST TO LOWEST)

- 1 Command line environment variables
- 2 environment in service definition
- 3 env_file (top to bottom order)
- 4 .env file (for interpolation only)
- 5 Defaults in the Compose file

COMMON .ENV EXAMPLE

```
APP_NAME=MyApp
NODE_ENV=production
APP_TAG=1.2.3
APP_PORT=8080
DB_USER=appuser
DB_PASSWORD=strongpassword
DB_HOST=db
DB_PORT=5432
REDIS_URL=redis://redis:6379
```

TIPS

- ✓ Never commit real secrets to Git.
- ✓ Use secrets for sensitive data.
- ✓ Keep .env files out of images.
- ✓ Audit and rotate credentials regularly.
- ✓ Automate validation in CI/CD pipelines.



SECRETS AND SENSITIVE CONFIGURATION

PROTECT YOUR SECRETS. SECURE YOUR APPLICATION.

1. COMPOSE SECRETS



- Store sensitive data separately from images.
- Mounted in memory inside containers.
- Not exposed in env or process list.
- Supported by Docker Compose and Swarm.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    secrets:
      - db_password
secrets:
  db_password:
    file: ./secrets/db_password.txt
```

2. FILE-BASED SECRETS



- Keep secret values in files outside Compose file.
- Restrict file permissions (600).
- Do not commit secret files to Git.

EXAMPLE (File Structure)

```
project/
├── docker-compose.yml
└── secrets/
    └── db_password.txt (600)
```

EXAMPLE (docker-compose.yml)

```
secrets:
  db_password:
    file: ./secrets/db_password.txt
```

3. ENVIRONMENT VARIABLE RISKS



- Visible in 'docker inspect'.
- May appear in logs.
- Can be exposed via process lists.
- Harder to audit and rotate.

AVOID THIS

```
services:
  app:
    image: myapp:latest
    environment:
      - DB_PASSWORD=supersecret
```



Environment variables are easier to leak than secrets.

4. PREVENTING SECRETS FROM ENTERING IMAGES



- Never use ARG or ENV for secrets.
- Use secrets or mounts at runtime.
- Multi-stage builds: never COPY secrets.
- Keep secrets out of Docker layers.

BEST PRACTICES

- ✓ Use secrets for runtime injection.
- ✓ Do not bake secrets into images.
- ✓ Do not echo secrets in RUN commands.
- ✓ Use .dockerignore to exclude secret files.

5. GIT REPOSITORY EXPOSURE



- Never commit secrets to Git.
- Avoid committing .env files.
- Use .gitignore to protect sensitive files.
- Review commit history regularly.

ADD TO .GITIGNORE

```
.env
secrets/
*.secret
*.key
*.pem
.env.*
```

WHAT NOT TO COMMIT

- ✗ .env files with real values
- ✗ Secret files and keys
- ✗ Database dumps
- ✗ Certificates
- ✗ API keys and tokens
- ✗ Cloud credentials



ONCE A SECRET IS IN GIT, IT IS IN PUBLIC HISTORY FOREVER.

6. SECRET ROTATION CONSIDERATIONS



- Rotate secrets regularly.
- Automate rotation when possible.
- Minimize secret lifetime.
- Update all dependent services.
- Test rotation in staging first.

TYPICAL ROTATION FLOW

- 1 Generate new secret
- 2 Update secret in secret store/file
- 3 Reload or restart dependent services
- 4 Verify application works
- 5 Revoke old secret
- 6 Audit and monitor

7. PRODUCTION SECURITY PATTERNS



- Use least privilege access.
- Isolate secret files.
- Encrypt at rest and in transit.
- Use secrets management tools if possible.
- Monitor and audit access.

SECURE PATTERNS

- ✓ Use Docker secrets or external secret stores.
- ✓ Inject secrets at runtime only.
- ✓ Restrict file permissions (600).
- ✓ Use read-only mounts for secrets.
- ✓ Limit secret access to only required services.
- ✓ Centralize and audit secret usage.

HOW COMPOSE SECRETS WORK



SECRET FILE
(Outside Project)



COMPOSE FILE
References Secret



DOCKER ENGINE
Stores Secret Securely



CONTAINER
Secret Mounted in Memory
(e.g., /run/secrets/secret_name)



APPLICATION
Reads Secret at Runtime



SECRETS STAY OUT OF IMAGES, ENV VARIABLES, AND LOGS.
ONLY AVAILABLE TO CONTAINERS THAT NEED THEM.



MOUNT POINT

/run/secrets/<secret_name>
(Readable only by root by default)

SERVICE DEPENDENCIES AND STARTUP ORDER

BUILD RELIABLE STARTUP SEQUENCES. ENSURE SERVICES ARE READY.

1. DEPENDS_ON



- Defines service dependencies.
- Controls startup order.
- Does not wait for readiness by default.
- Available in Docker Compose file format.

EXAMPLE (docker-compose.yml)

services:

web:

```
image: myapp:latest
depends_on:
  - db
  - redis
```

db:

```
image: postgres:15
```

redis:

```
image: redis:7
```

2. DEPENDENCY CONDITIONS



- Control how Compose waits for dependencies.
- Use conditions with long-form depends_on.
- Three main conditions available.

CONDITIONS

- ✓ **service_started**
– Dependency container has started.
- ✓ **service_healthy**
– Dependency container is healthy.
- ✓ **service_completed_successfully**
– Dependency container exited with code 0.

EXAMPLE (Long Syntax)

```
depends_on:
  db:
    condition: service_healthy
  migrate:
    condition: service_completed_successfully
```

3. STARTUP VS READINESS



- Container started ≠ Service ready.
- Applications need time to initialize.
- Always check readiness before connecting.

STARTUP (Container Running)

The process is running but the service may not be ready.



READINESS (Service Ready)

The service can accept connections and handle requests.



4. HEALTH CHECKS



- Health checks tell Compose when a service is healthy.
- Used with condition: service_healthy.
- Essential for production reliability.

EXAMPLE (Health Check)

services:

db:

```
image: postgres:15
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 20s
```

5. DATABASE READINESS



- Databases take time to initialize.
- Wait for connections to be ready.
- Use health checks + depends_on with service_healthy.
- Verify with real connection tests.

EXAMPLE (PostgreSQL)

services:

db:

```
image: postgres:15
environment:
  POSTGRES_PASSWORD: strongpass
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 20s
```

6. RETRY LOGIC



- Applications should retry on failure.
- Handles temporary unavailability.
- Use exponential backoff.
- Improves resilience.

EXAMPLE (App Retry Logic)

```
max_retries = 10
delay = 2
for attempt in range(max_retries):
  try:
    connect_to_db()
    break
  except ConnectionError:
    sleep(delay)
    delay *= 2
```

7. WHY STARTUP ORDER ALONE IS INSUFFICIENT



- depends_on only controls startup sequence.
- It does NOT wait for services to be ready.
- Race conditions can still cause failures.
- Always combine with health checks and retry logic.

8. DEPENDENCY FAILURE PATTERNS



- Database not ready.
- Dependency crash loops.
- Network not available.
- Configuration errors.
- Timeouts on connection.
- Resource exhaustion.

MITIGATION

- ✓ Health checks
- ✓ Retry with backoff
- ✓ Timeouts
- ✓ Circuit breaker pattern
- ✓ Monitoring & alerts

DEPENDENCY FLOW EXAMPLE



BEST PRACTICES

- ✓ Use health checks for all stateful services.
- ✓ Use depends_on with condition: service_healthy.
- ✓ Implement retry logic with exponential backoff.
- ✓ Test failure scenarios during development.
- ✓ Monitor dependencies in production.

QUICK REFERENCE

depends_on	Controls startup order
service_started	Container has started
service_healthy	Container is healthy
service_completed_successfully	Container exited successfully
Health Check	Defines readiness
Retry Logic	Handles temporary failures

COMMON PITFALLS

- ✗ Relying on startup order only
- ✗ No health checks
- ✗ Hardcoded sleep/wait scripts
- ✗ Ignoring dependency failures
- ✗ No retry or timeout handling
- ✗ Not testing failure scenarios



HEALTH CHECKS AND APPLICATION READINESS

DETECT PROBLEMS EARLY. KEEP YOUR SERVICES RELIABLE.

1. HEALTHCHECK



- Defines a command Docker runs to check container health.
- Essential for production reliability.
- Works with Compose and Docker Swarm.

EXAMPLE (docker-compose.yml)
healthcheck:
test: ["CMD", "curl", "-f",
"http://localhost:8080/health"]

2. TEST



- The command used to check health.
- Must exit 0 on success.
- Common types: CMD, CMD-SHELL, or NONE.

EXAMPLES
• CMD: ["CMD", "pg_isready", "-U",
"postgres"]
• CMD-SHELL: curl -f http://localhost:8080/health

3. INTERVAL



- How often the health check runs.
- Too low = overhead.
- Too high = slow detection.

EXAMPLE
interval: 10s

4. TIMEOUT



- Maximum time allowed for the check to complete.
- If exceeded, the check fails.

EXAMPLE
timeout: 5s

5. RETRIES



- Number of consecutive failures before the container is marked unhealthy.
- Higher = more tolerant.

EXAMPLE
retries: 5s

6. START_PERIOD



- Grace period before health checks start counting failures.
- Prevents false negatives during startup.

EXAMPLE
start_period: 30s

7. DESIGNING MEANINGFUL HEALTH ENDPOINTS



- Return 200 OK only when the service is truly ready.
- Check dependencies (DB, cache, queue).
- Keep checks lightweight and fast.
- Separate liveness (/health) and readiness (/ready) when possible.
- Avoid expensive operations.

8. HEALTH CHECK EXAMPLES

SERVICE	TEST EXAMPLE
WEB APP (HTTP)	curl -f http://localhost:8080/health
NODE.JS APP	node -e "require('http').get('http://localhost:3000/health', r => process.exit(r.statusCode === 200 ? 0 : 1))"
SPRING BOOT	curl -f http://localhost:8080/actuator/health
POSTGRES	pg_isready -U postgres
MYSQL	mysqladmin ping -h 127.0.0.1 -u root -p\$5
REDIS	redis-cli ping
RABBITMQ	rabbitmq-diagnostics -q ping

9. COMPOSE HEALTHCHECK EXAMPLE

```
services:  
  web:  
    image: myapp:latest  
    healthcheck:  
      test: ["CMD", "curl", "-f",  
        "http://localhost:8080/health"]  
      interval: 10s  
      timeout: 5s  
      retries: 3  
      start_period: 30s
```

10. DIAGNOSING UNHEALTHY CONTAINERS



- ### 1. CHECK STATUS
- See if the container is unhealthy.

```
docker ps
```

CONTAINER ID	NAME	STATUS
ab12cd34	web	unhealthy (2) 10s ago



- ### 2. INSPECT HEALTH DETAILS
- View detailed health check output.

```
docker inspect --format='{{.json.State.Health}}'  
<container_id>  
{  
  "Status": "unhealthy",  
  "FailingStreak": 2,  
  "Log": [  
    {  
      "Start": "2024-05-20T12:00:00Z",  
      "End": "2024-05-20T12:00:01Z",  
      "ExitCode": 1,  
      "Output": "curl: (7) Failed to connect"    }  
  ]  
}
```



- ### 3. VIEW CONTAINER LOGS
- Check application logs for errors.

```
docker logs -f <container_id>
```

```
INFO Starting application...  
INFO Connecting to database...  
ERROR Connection refused  
ERROR Database unavailable
```



- ### 4. EXEC INTO CONTAINER
- Debug from inside the container.

```
docker exec -it <container_id> sh
```

```
/ # curl -v http://localhost:8080/health  
* Trying 127.0.0.1:8080...  
* connect to 127.0.0.1 port 8080 failed:  
Connection refused  
/ # nc -zv db 5432  
nc: connect to db (172.18.0.2) 5432  
(tcp) failed: Connection refused
```

11. BEST PRACTICES

- Always implement health checks for all stateful services.
- Use meaningful endpoints that check real dependencies.
- Set appropriate intervals and timeouts (not too aggressive).
- Use start_period to avoid premature failures.
- Monitor health status in production (alerts & dashboards).
- Combine health checks with retries in applications.

12. COMMON HEALTH CHECK ISSUES

ISSUE	CAUSE	SOLUTION
Always unhealthy	Wrong command or endpoint	Verify command and endpoint return 200
Flapping (up/down)	Interval too low or check unstable	Increase interval and stabilize check
Slow to detect failure	Interval or retries too high	Reduce interval or retries
Startup failures	No start_period configured	Add start_period for slow startups
False positives	Check too simple (always returns 200)	Implement meaningful checks

13. HEALTH CHECK LIFECYCLE



CONTAINER STARTS
Container is running



START_PERIOD
Grace period
(no failures counted)



HEALTH CHECKS RUN
Check is executed
at every interval



SUCCESS
Service is
healthy



FAILURES INCREASE
Failing streak increments
on each failure



MARKED UNHEALTHY
After retries exceeded,
container is unhealthy

RESTART POLICIES AND FAILURE RECOVERY

AUTOMATIC RECOVERY. MINIMAL DOWNTIME. MAXIMUM RELIABILITY.

1. RESTART: NO



- Never restart the container automatically.
- Manual restart only.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    restart: "no"
```

2. RESTART: ALWAYS



- Always restart the container if it stops.
- Restarts on any exit, success or failure.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    restart: "always"
```

3. RESTART: ON-FAILURE



- Restart only if the container exits with a failure (non-zero code).
- Optional: limit number of retries.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    restart: on-failure:5

    (Retry max 5 times)
```

4. RESTART: UNLESS-STOPPED



- Always restart unless the container was stopped manually.

EXAMPLE (docker-compose.yml)

```
services:
  app:
    image: myapp:latest
    restart: unless-stopped
```

5. MANUAL STOP BEHAVIOR



- "always" and "unless-stopped" won't restart if stopped manually.
- "on-failure" still respects failure rules.

6. CRASH LOOPS



- Container crashes immediately after start.
- Restart policy keeps restarting it.
- Causes high CPU, logs flooding.

DETECTION

docker ps
Look for containers with STATUS: "Restarting (n seconds ago)"

docker logs <container>
Check logs for root cause.

7. DEPENDENCY FAILURES



- Service starts before its dependency is ready.
- Connection fails.
- May trigger restarts or crash loops.

COMMON CAUSES

- Database not ready
- Cache not initialized
- Missing DNS / network
- External service unavailable

SOLUTION: Use healthchecks, depends_on with conditions, and application retry logic.

8. APPLICATION-LEVEL RETRY BEHAVIOR



- Handle transient errors in code.
- Use exponential backoff.
- Set reasonable retry limits.
- Fail fast on non-recoverable errors.

RETRY STRATEGY EXAMPLE

- Initial delay: 1s
- Backoff: x2
- Max delay: 30s
- Max attempts: 5
- Jitter: Add randomness

9. CHOOSING THE CORRECT RECOVERY STRATEGY



CONSIDER:

- ✓ Is the failure temporary or permanent?
- ✓ Can the service recover automatically?
- ✓ Is manual intervention required?
- ✓ Will restarts cause more harm?
- ✓ Do dependencies need time to recover?

10. RESTART POLICY COMPARISON

POLICY	RESTARTS WHEN	RESTARTS AFTER MANUAL STOP	ON SUCCESS EXIT (EXIT 0)	ON FAILURE EXIT (EXIT ≠ 0)	USE CASE
no	Never automatically	✗	✗	✗	Jobs, one-off tasks, manual control
always	Any exit (success or failure)	✗	✓	✓	Critical services, high availability
on-failure[:max]	Only on failure (non-zero exit)	✗	✗	✓ (up to max)	Apps that may fail but can recover
unless-stopped	Any exit (success or failure)	✓	✓	✓	Services that should always restart

11. RECOVERY FLOW



FAILURE OCCURS
Container exits with an error.



RESTART POLICY EVALUATED
Docker checks the restart rule.



CONTAINER RESTARTED
Container is started again.



SERVICE RECOVERS
Application becomes healthy.

12. COMMON PITFALLS

- ✗ Using "always" for jobs or one-time tasks.
- ✗ Ignoring crash loops and log flooding.
- ✗ No healthchecks + no dependency conditions.
- ✗ Relying only on restart policy without fixing root cause.
- ✗ Not limiting retries in on-failure policy.

13. BEST PRACTICES

- ✓ Use healthchecks for all critical services.
- ✓ Use depends_on with conditions.
- ✓ Implement retry logic with backoff in applications.
- ✓ Use on-failure with a retry limit when appropriate.
- ✓ Monitor restarts and alert on repeated failures.

14. MONITOR WHAT MATTERS



Restart count



Crash loops



Service uptime



Health status



RESTART POLICIES HELP RECOVER,
BUT SOLID APPLICATION DESIGN PREVENTS FAILURE.



DESIGN FOR FAILURE.
BUILD FOR RESILIENCE.

RESOURCE MANAGEMENT

CONTROL CONTAINER RESOURCES. PROTECT THE HOST. DELIVER STABILITY.

1 CPU LIMITS



- Limit the maximum CPU a container can use.
- Prevents noisy neighbor problems.
- CPU is measured in cores (e.g., 500m = 0.5 CPU).
- If not set, container can use all available CPU.
- Use limits for predictable performance.

2 MEMORY LIMITS



- Limit the maximum memory a container can use.
- Prevents Out-Of-Memory (OOM) kills.
- Memory is measured in MiB/GiB.
- If exceeded, the container is killed.
- Set limits for stability.

3 RESERVATIONS (REQUESTS)



- Define the minimum CPU and memory a container needs.
- Scheduler uses requests to place Pods.
- Ensures resources are available for the Pod.
- Set requests based on actual usage.

4 OUT-OF-MEMORY FAILURES



- Occurs when memory limit is exceeded.
- Kernel OOM killer terminates the container.
- Causes restarts and application failures.
- Monitor memory usage closely.
- Set proper limits and alerts.

5 RESOURCE CONTENTION



- Multiple containers compete for limited resources.
- Leads to high latency and poor performance.
- CPU throttling slows down containers.
- Memory pressure can cause OOMs.
- Use limits, requests and proper sizing.

6 CONTAINER RESOURCE VISIBILITY



- Use metrics to understand usage.
- Tools: Metrics Server, Prometheus, CloudWatch.
- Monitor CPU, memory, network, disk.
- Set dashboards and alerts.
- Visibility enables better decisions.

7 CAPACITY PLANNING



- Forecast future resource needs.
- Consider growth, seasonality and peaks.
- Use historical metrics and trends.
- Ensure enough node capacity.
- Avoid over-provisioning and under-provisioning.

8 PREVENTING ONE SERVICE FROM EXHAUSTING THE HOST



- Set CPU and memory limits for all containers.
- Use requests for guaranteed resources.
- Enable ResourceQuotas and LimitRanges.
- Isolate critical workloads with nodes/taints.
- Enforce policies and monitor continuously.

RESOURCE TYPES AND UNITS

RESOURCE	DESCRIPTION	UNIT	EXAMPLE	KUBERNETES FIELD
CPU	Processing power	Cores (millicores)	500m = 0.5 CPU	resources.limits.cpu
MEMORY	RAM usage	MiB, GiB	512Mi, 1Gi, 2Gi	resources.limits.memory
EPHEMERAL STORAGE	Temporary storage	GiB	10Gi	resources.limits.ephemeral-storage

EXAMPLE: COMPOSE RESOURCE CONFIG

```
services:
  api:
    image: my-api:1.0
    deploy:
      resources:
        limits:
          cpus: '1.0'           # Max 1 CPU core
          memory: 1Gi          # Max 1 GiB memory
        reservations:
          cpus: '0.5'          # Min 0.5 CPU core
          memory: 512Mi        # Min 512 MiB memory
  worker:
    image: my-worker:1.0
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 512Mi
        reservations:
          cpus: '0.25'
          memory: 256Mi
```

HOW RESOURCE MANAGEMENT PROTECTS YOUR SYSTEM



LIMITS & REQUESTS
APPLIED



FAIR SCHEDULING
& ISOLATION



PREDICTABLE
PERFORMANCE



STABLE APPLICATIONS
& HOST PROTECTION

COMMON PROBLEMS AND PREVENTION

PROBLEM	CAUSE	IMPACT	PREVENTION
CPU THROTTLING	CPU limit too low	High latency, slow response	Set proper CPU limits and monitor usage
OOM KILLED	Memory limit too low	Container restart, downtime	Increase memory limit and optimize app
NODE OVERLOAD	No limits or requests	All services slow or fail	Set limits, requests and quotas
POOR SCHEDULING	Requests not set	Pods may not get enough resources	Always set requests based on usage

BEST PRACTICES CHECKLIST



SET CPU & MEMORY
LIMITS FOR ALL
SERVICES



SET REQUESTS BASED
ON ACTUAL USAGE



MONITOR RESOURCE
METRICS CONTINUOUSLY



ALERT ON HIGH USAGE
AND THROTTLING



USE RESOURCEQUOTAS
AND LIMITRANGES



ISOLATE CRITICAL
WORKLOADS



REVIEW AND ADJUST
REGULARLY

BUILDING IMAGES WITH COMPOSE



BUILD FAST. BUILD SMART. BUILD REPRODUCIBLE IMAGES.

1. BUILD



Use the build key to build an image from a Dockerfile.
Better for custom applications.

2. BUILD CONTEXT



Defines the directory sent to the Docker daemon.
Only files inside the context are available during build.

3. DOCKERFILE SELECTION



Specify the Dockerfile path if not named Dockerfile.
Keep Dockerfiles clean and specific.

4. BUILD ARGUMENTS



Pass variables at build time using args.
Useful for versions, tokens, and build customization.

5. MULTI-STAGE BUILDS



Use multi-stage builds to reduce image size.
Keep for versions, small, secure, and efficient.

6. IMAGE TAGS



Tag images for version control and traceability.
Use semantic tags and commit SHAs.

7. BUILD CACHE



Compose leverages Docker build cache.
Speeds up builds by reusing unchanged layers.

8. REPRODUCIBLE BUILDS



Pin base images.
Avoid "latest" tags.
Use exact versions for consistent builds.

9. SEPARATING BUILD-TIME & RUNTIME CONFIG



Use ARG for build-time values.
Use ENV for runtime configuration.
Keep secrets out of images.

COMPOSE BUILD EXAMPLE

```
services:
  api:
    build:
      context: ./api           # Build context
      dockerfile: Dockerfile   # Dockerfile path
      target: production        # Multi-stage target
      args:
        APP_VERSION: ${APP_VERSION} # Build argument
        GIT_COMMIT: ${GIT_COMMIT}    # Build argument
    image: myorg/api:${APP_VERSION} # Image tag
    ports:
      - "8080:8080"             # Published port
    environment:
      - NODE_ENV=production      # Runtime env vars
      - LOG_LEVEL=info
    volumes:
      - ./api/uploads:/app/uploads # Persistent data
    networks:
      - backend                  # Attach to network
  backend:
    driver: bridge
```

DOCKERFILE MULTI-STAGE EXAMPLE

```
# Stage 1: Build
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/dist ./dist
COPY package*.json .
RUN npm ci --omit=dev
EXPOSE 8080
CMD ["node", "dist/server.js"]
```

BUILD COMMANDS

- ✓ `docker compose build`
Build all services
- ✓ `docker compose build api`
Build a specific service
- ✓ `docker compose build --no-cache`
Build without using cache
- ✓ `docker compose build --pull`
Always pull latest base images

BEST PRACTICES

- ✓ Use specific base image versions.
- ✓ Keep Dockerfiles small and focused.
- ✓ Leverage `.dockerignore` to reduce context size.
- ✓ Use multi-stage builds for smaller images.
- ✓ Separate build-time args from runtime envs.
- ✓ Tag images with version and commit.
- ✓ Cache dependencies before copying source.
- ✓ Do not bake secrets into images.

BENEFITS



FASTER BUILDS
Leverage cache and smaller contexts.



SMALLER IMAGES
Reduce size and attack surface.



REPRODUCIBLE CONSISTENT BUILDS
Same source = same image.



SECURE & CLEAN
No secrets and only runtime config at runtime.

KEY TAKEAWAY

**COMPOSE MAKES IMAGE BUILDING SIMPLE, REPEATABLE, AND PRODUCTION-READY.
GOOD DOCKERFILES + SMART CONFIG = RELIABLE DEPLOYMENTS.**

COMPOSE OVERRIDES AND MULTIPLE ENVIRONMENTS



ONE BASE. MANY ENVIRONMENTS. ZERO DUPLICATION.

1. MULTIPLE COMPOSE FILES



Compose supports multiple yaml files.

Split base configuration from environment-specific settings.

Load multiple files in order.

2. CONFIGURATION MERGING



Compose merges files from left to right.

Later files override earlier ones.

Merge is deep and intelligent.

3. DEVELOPMENT OVERRIDES



Add local tools, volume mounts, and debuggers.

Use hot-reload and bind mounts.

Keep dev-only settings out of production.

4. TESTING ENVIRONMENTS



Use isolated test configs and databases.

Enable extra logging and debugging.

Reset data between test runs.

5. STAGING CONFIGURATION



Mirror production as closely as possible.

Use production-like resources and settings.

Enable monitoring and health checks.

6. PRODUCTION CONFIGURATION



Use secure defaults and minimal services.

No debug tools or bind mounts.

Use secrets, limits, and optimized settings.

7. AVOIDING DUPLICATED YAML



Keep common settings in base compose.yaml.

Override only what changes.

Use anchors and extensions when needed.

8. ENVIRONMENT PROMOTION PATTERNS



Promote the same artifact across environments.

Change only configuration, not code.

Ensure consistency and reliability at every stage.

HOW COMPOSE FILES WORK TOGETHER

FILE LAYERING ORDER (LEFT TO RIGHT)



EXAMPLE COMMANDS

- `docker compose up -f compose.yaml`
- `docker compose -f compose.yaml -f compose.override.yaml up`
- `docker compose -f compose.yaml -f compose.staging.yaml up`
- `docker compose -f compose.yaml -f compose.prod.yaml up`

EXAMPLE FILE STRUCTURE



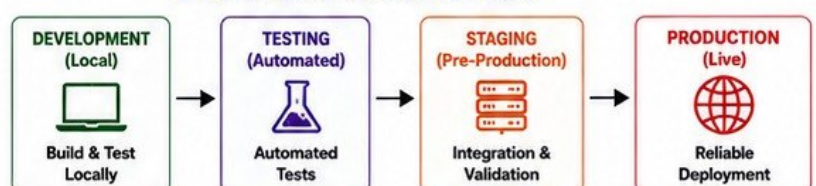
WHAT SHOULD CHANGE PER ENVIRONMENT?

SETTING	DEVELOPMENT	TESTING	STAGING	PRODUCTION
DEBUG / LOG LEVEL	DEBUG	INFO / DEBUG	INFO	WARN / ERROR
VOLUMES	BIND MOUNTS	NAMED VOLUMES	NAMED VOLUMES	NAMED VOLUMES
DATABASE	LOCAL DB	TEST DB	STAGING DB	PRODUCTION DB
ENVIRONMENT VARIABLES	.env (local)	.env.test	.env.staging	.env.prod
REPLICAS	1	1	2-3	3+
RESOURCE LIMITS	NONE	LIGHT	MEDIUM	STRICT
MONITORING	MINIMAL	BASIC	ENABLED	FULL

BEST PRACTICES

- ✓ Keep compose.yaml clean and environment-agnostic.
- ✓ Override only what is necessary.
- ✓ Use multiple files instead of large, complex single files.
- ✓ Use .env files for all environment-specific variables.
- ✓ Never commit secrets or sensitive data.
- ✓ Test your configuration merging in CI/CD.
- ✓ Promote the same image across all environments.

ENVIRONMENT PROMOTION FLOW





PROFILES AND OPTIONAL SERVICES

RUN ONLY WHAT YOU NEED. KEEP THE REST OPTIONAL.

1. COMPOSE PROFILES



- Profiles group services that are not always needed.
- Services start only when their profile is activated.
- Keep the default startup minimal and clean.

2. SELECTIVE SERVICE STARTUP



- Start only required services for the current task.
- Faster startup and lower resource usage.
- Example: run app without monitoring or debug tools.

3. DEBUGGING TOOLS



- Include tools like curl, bash, or admin UI.
- Activate only when debugging issues.
- Keep production images small and secure.

4. DEVELOPMENT-ONLY SERVICES



- Hot reload servers, mail catchers, and mock APIs.
- Never run in staging or production.
- Improve developer experience without impacting prod.

5. MONITORING SERVICES



- Metrics, logs, and alerting stacks.
- Activate in dev, staging, or prod as needed.
- Keep monitoring isolated and optional.

6. MIGRATION JOBS



- Run one-time jobs for schema or data migrations.
- Execute on demand, not on every start.
- Keep main services unaffected.

7. KEEPING OPTIONAL WORKLOADS OUT OF NORMAL STARTUP



- Use profiles to separate concerns.
- Ensure stable, predictable, and fast default boot.
- Optional workloads run only when explicitly enabled.

EXAMPLE: COMPOSE FILE WITH PROFILES

```
services:
  app:
    image: myorg/app:latest
    ports:
      - "8080:8080"
    depends_on:
      - db
    profiles: ["default"] # Always started
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: appdb
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
    profiles: ["default"] # Always started
  adminer:
    image: adminer
    ports:
      - "8081:8080"
    profiles: ["debug"] # Debugging tool
  mailhog:
    image: mailhog/mailhog
    ports:
      - "8025:8025"
    profiles: ["dev"] # Dev only service
  prometheus:
    image: prom/prometheus
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
    profiles: ["monitoring"] # Monitoring stack
  migrate:
    image: myorg/app:latest
    command: ["npm", "run", "migrate"]
    depends_on:
      - db
    profiles: ["migration"] # Run on demand
volumes:
  db_data: # Persistent database data
```

HOW PROFILES WORK

COMPOSE FILE
(ALL SERVICES)



ACTIVATE
PROFILES



START ONLY
SELECTED
SERVICES



COMMON PROFILES EXAMPLES

PROFILE	TYPICAL SERVICES
default	Core application, database, cache
debug	Admin tools, CLI containers, databases UI
dev	Hot reload, mail catcher, mock services
monitoring	Prometheus, Grafana, Loki, Alertmanager
migration	Database migrations, one-time jobs
backup	Backup jobs, export tools
test	Test runners, coverage tools

START SERVICES WITH PROFILES

docker compose up	Start default profile services only
docker compose --profile debug up	Start default + debug services
docker compose --profile dev up	Start default + dev services
docker compose --profile monitoring up	Start with monitoring stack
docker compose --profile migration run migrate	Run migration job only

BEST PRACTICES



KEEP DEFAULT
PROFILE MINIMAL
AND STABLE.



GROUP RELATED
SERVICES UNDER
THE SAME PROFILE.



NEVER INCLUDE
DEV OR DEBUG
TOOLS IN PROD.



USE PROFILES TO
IMPROVE SPEED
AND RELIABILITY.



DOCUMENT PROFILES
AND THEIR PURPOSE
FOR YOUR TEAM.



AUTOMATE PROFILE
USAGE IN CI/CD
PIPELINES.



LOGGING AND OBSERVABILITY

SEE EVERYTHING. UNDERSTAND EVERYTHING. FIX FASTER.

1. CONTAINER LOGS



- Every container generates logs.
- STDOUT and STDERR are captured.
- Logs are ephemeral by default.
- Critical for debugging and monitoring.

2. DOCKER COMPOSE LOGS



- View logs for one or all services.
- Follow logs in real time.
- Filter by service, time, or since.
- Combine with timestamps.

3. LOG DRIVERS



- Control how logs are captured and stored.
- Use drivers to send logs to external systems.
- Improve retention and performance.

4. STRUCTURED APPLICATION LOGS



- Use JSON or key-value structured logs.
- Easier to parse, search, and alert on.
- Include context: request id, user id, trace id, etc.

5. CENTRALIZED LOGGING



- Aggregate logs in a central system.
- Better search, alerting, and dashboards.
- Long-term retention and compliance.

6. HEALTH INFORMATION



- Health checks provide runtime status.
- Inspect container health via Compose.
- Use health info for alerts and automation.

7. RESOURCE MONITORING



- Monitor CPU, memory, disk, and network.
- Detect bottlenecks and abnormal usage.
- Use metrics for capacity planning.

8. TROUBLESHOOTING WITH CORRELATED LOGS



- Correlate logs across services.
- Use timestamps and request/trace ids.
- Reconstruct full request flow.
- Find root cause faster.

9. BEST PRACTICES



- Log important events only.
- Use structured logs.
- Centralize and retain logs.
- Monitor health and resources.
- Correlate logs during incidents.

DOCKER COMPOSE LOG COMMANDS

```
# View logs for all services
docker compose logs

# View logs for a specific service
docker compose logs api

# Follow logs in real time
docker compose logs -f

# Follow logs for a service
docker compose logs -f api

# Show timestamps
docker compose logs -t

# Show logs since 10 minutes ago
docker compose logs --since 10m

# Tail last 100 lines
docker compose logs --tail 100

# Combine follow + timestamps + service
docker compose logs -f -t api

# Filter by multiple services
docker compose logs -f api worker db
```

EXAMPLE: LOGGING CONFIGURATION (COMPOSE)

```
services:
  api:
    image: myorg/api:latest
    logging:
      driver: "json-file"           # Default driver
      options:
        max-size: "10m"           # Rotate at 10MB
        max-file: "5"             # Keep 5 files
  web:
    image: myorg/web:latest
    logging:
      driver: "gelf"               # Send to Graylog
      options:
        gelf-address: "udp://graylog:12201"
        tag: "web"
  worker:
    image: myorg/worker:latest
    logging:
      driver: "fluentd"            # Send to Fluentd
      options:
        fluentd-address: "fluentd:24224"
        tag: "worker"
```

COMMON LOG DRIVERS

DRIVER	DESCRIPTION
json-file	Default. Logs stored locally in JSON files.
syslog	Send logs to syslog server.
gelf	Send logs to Graylog.
fluentd	Send logs to Fluentd.
awslogs	Send logs to AWS CloudWatch Logs.
splunk	Send logs to Splunk.
local	High-performance local logging (Docker).
none	Disable logging (Not recommended).

HEALTH CHECK EXAMPLE

```
services:
  api:
    image: myorg/api:latest
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
```

HEALTH STATUS COMMANDS

```
docker compose ps           # Show health status
docker inspect <container> # Detailed health info
```

RESOURCE MONITORING COMMANDS

```
# Live resource usage
docker stats

# Stats for specific services
docker compose top

# Inspect container resource limits
docker inspect <container> | grep -A 10 HostConfig

# Check events (restarts, OOM, etc.)
docker events --filter container=<container>
```

TROUBLESHOOTING CHECKLIST

- ✓ Check service health: `docker compose ps`
- ✓ Review recent logs: `docker compose logs --since 10m`
- ✓ Correlate timestamps across services
- ✓ Look for errors, warnings, and exceptions
- ✓ Check resource usage: `docker stats`
- ✓ Verify network and dependency health
- ✓ Confirm environment and configuration
- ✓ Reproduce and observe with detailed logs



APPLICATION
GENERATES LOGS



CONTAINER
CAPTURES LOGS



LOG DRIVER
PROCESSES LOGS



CENTRALIZED
LOG SYSTEM



MONITOR, SEARCH,
ALERT & ANALYZE



FASTER INCIDENT
DETECTION & RESOLUTION



PRODUCTION SECURITY HARDENING

SECURE BY DESIGN. LEAST PRIVILEGE. DEFENSE IN DEPTH.



1. NON-ROOT CONTAINERS

- Run containers as non-root users.
- Prevent privilege escalation inside containers.
- Set USER in Dockerfile and Compose.



2. READ-ONLY FILESYSTEMS

- Mount the root filesystem as read-only.
- Prevent unwanted changes and malware persistence.
- Use read_only: true.



3. LINUX CAPABILITIES

- Drop unnecessary kernel capabilities.
- Add only what the application truly needs.
- Use cap_drop and cap_add wisely.



4. SECURITY_OPT (LSM PROFILES)

- Use AppArmor or SELinux profiles to restrict behavior.
- Apply additional kernel-level security policies.
- Example: no-new-privileges.



5. AVOID PRIVILEGED CONTAINERS

- Never run containers in --privileged mode.
- It bypasses most isolation and security controls.
- Use granular permissions instead.



6. NETWORK EXPOSURE

- Publish only necessary ports.
- Use internal networks for service-to-service traffic.
- Restrict access with firewalls and security groups.



7. SECRET HANDLING

- Use Docker secrets or external secret stores.
- Never store secrets in images or environment files.
- Rotate secrets regularly.



8. IMAGE SECURITY

- Use minimal base images.
- Scan images for vulnerabilities.
- Keep images updated and rebuild regularly.



9. LEAST-PRIVILEGE COMPOSE CONFIG

- Apply the minimum permissions required to run.
- Combine all security controls for defense in depth.
- Review and audit your Compose file regularly.

SECURE DOCKER COMPOSE EXAMPLE

```

compose.
yaml
services:
  web:
    image: myapp:latest
    user: "1000:1000"
    read_only: true
    tmpfs:
      - /tmp
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
      - apparmor:docker-default
    ports:
      - "127.0.0.1:8080:8080"
    environment:
      - APP_ENV=production
    secrets:
      - db_password
    networks:
      - app_net

secrets:
  db_password:
    file: ./secrets/db_password.txt

networks:
  app_net:
    driver: bridge
    internal: true
  
```

Run as non-root user

Read-only root filesystem

Drop all unnecessary capabilities

Enforce security options (no-new-privileges, AppArmor)

Bind ports only to localhost

Use secrets instead of environment variables

Use internal network for isolation



PRODUCTION SECURITY CHECKLIST

- ✓ Run as non-root user
- ✓ Root filesystem read-only
- ✓ Drop unnecessary capabilities
- ✓ Apply security_opt profiles
- ✓ Avoid privileged containers
- ✓ Expose only required ports
- ✓ Use internal networks
- ✓ Handle secrets securely
- ✓ Use minimal, updated images
- ✓ Follow least-privilege principle
- ✓ Review Compose file regularly



NEVER DO THIS

- ✗ Run as root
- ✗ Use --privileged
- ✗ Expose unnecessary ports
- ✗ Store secrets in images or env files
- ✗ Use outdated or vulnerable images



KEY TAKEAWAY

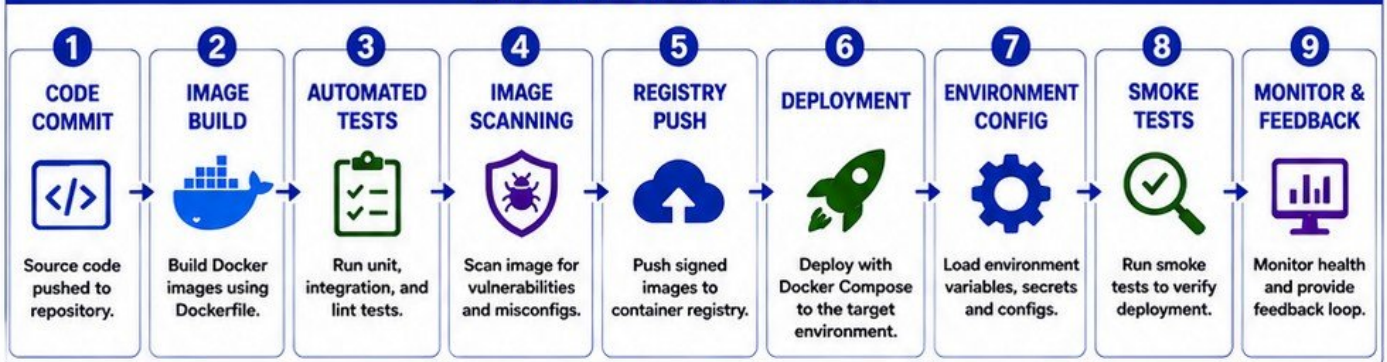
A SECURE COMPOSE APPLICATION IS BUILT ON LEAST PRIVILEGE, MINIMAL ATTACK SURFACE, AND STRONG ISOLATION.

CI/CD WITH DOCKER COMPOSE

AUTOMATE. TEST. SECURE. DEPLOY. REPEAT.



CI/CD PIPELINE ARCHITECTURE



- 1 CODE COMMIT**
- Developers push code to Git repository.
 - Triggers the CI/CD pipeline.
 - Use pull requests and branch protection.

- 2 IMAGE BUILD**
- Build Docker images with Dockerfile.
 - Use multi-stage builds for smaller images.
 - Tag with commit SHA and version.

- 3 AUTOMATED TESTS**
- Run unit tests.
 - Run integration tests using docker compose.
 - Enforce quality gates.
 - Fail pipeline on test errors.

- 4 IMAGE SCANNING**
- Scan image for CVEs using tools like Trivy or Grype.
 - Scan for secrets and misconfigurations.
 - Fail build on high or critical issues.

- 5 REGISTRY PUSH**
- Push approved images to container registry (Docker Hub, ECR, GHCR, etc).
 - Use immutable tags.
 - Sign images if possible.

- 6 DEPLOYMENT**
- Pull latest images.
 - Deploy using docker compose up -d.
 - Use Compose files optimized for target environment.

- 7 ENVIRONMENT CONFIGURATION**
- Use .env files, secrets, and configs.
 - Inject environment-specific variables.
 - Keep secrets out of Git.
 - Validate configuration before deployment.

- 8 SMOKE TESTS**
- Verify the application is running.
 - Check critical endpoints.
 - Validate dependent services.
 - Fail fast if unhealthy.

- 9 FAILED DEPLOYMENT HANDLING**
- Automatic rollback with previous working version.
 - Alert DevOps team.
 - Capture logs and metrics.
 - Provide clear failure notifications.

EXAMPLE COMPOSE DEPLOY COMMAND

```

# Pull latest images
docker compose pull

# Deploy in detached mode
docker compose up -d

# Remove unused resources
docker compose down --remove-orphans
  
```

BEST PRACTICES

- ✓ Automate everything.
- ✓ Fail fast and fix early.
- ✓ Scan every image.
- ✓ Use immutable images.
- ✓ Keep environments consistent.
- ✓ Test in an environment like production.
- ✓ Monitor continuously.
- ✓ Secure your pipeline.

PIPELINE TOOLS (EXAMPLES)

- GitHub Actions
- GitLab CI/CD
- Jenkins
- AWS CodePipeline
- Azure DevOps



GOAL: DELIVER SECURE, RELIABLE, AND REPEATABLE DEPLOYMENTS USING DOCKER COMPOSE AND A WELL-DESIGNED CI/CD PIPELINE.

DOCKER COMPOSE TROUBLESHOOTING



DIAGNOSE. ISOLATE. RESOLVE. DELIVER RELIABILITY.

1

CONTAINER EXITS IMMEDIATELY



- Check container logs.
- Verify command and endpoint.
- Validate environment variables.
- Check missing files or permissions.
- Use: `docker compose logs <svc>`

2

SERVICE CANNOT REACH ANOTHER SERVICE



- Verify service names and networks.
- Ensure both services are on the same network.
- Check firewall rules.
- Test connectivity with ping or curl.
- Use: `docker exec -it <svc> sh`

3

PORT CONFLICTS



- Host port already in use.
- Check running processes.
- Change host port in `compose.yaml`.
- Use: `lsof -i :<port>`
- Avoid publishing ports not needed.

4

DNS FAILURES



- Service name resolution failing.
- Verify network configuration.
- Check for custom DNS settings.
- Restart Docker if needed.
- Use: `docker network inspect <network>`

5

PERMISSION ERRORS



- Host volume permissions mismatch.
- Use correct user and group.
- Fix file ownership.
- Avoid running as root.
- Use: `chown -R <user>:<group> <path>`

6

VOLUME PROBLEMS



- Data not persisting.
- Wrong volume path.
- Named volume not found.
- Clear orphaned volumes.
- Use: `docker volume ls`
- Use: `docker compose down -v` (only if safe)

7

UNHEALTHY CONTAINERS



- Check healthcheck logs.
- Verify healthcheck command.
- Increase retries or timeouts.
- Ensure dependencies are ready.
- Use: `docker inspect <svc>`

8

CONFIGURATION VALIDATION



- Validate `compose.yaml` syntax.
- Check indentation and format.
- Validate required fields.
- Use: `docker compose config`
- Use: `docker compose config --quiet`

9

SYSTEMATIC TROUBLESHOOTING WORKFLOW



- Reproduce the issue.
- Check logs and events.
- Isolate the failing component.
- Verify configuration.
- Test hypotheses.
- Apply fix and validate.
- Document the resolution.

SYSTEMATIC TROUBLESHOOTING WORKFLOW

1
REPRODUCE THE ISSUE

Understand when and how it happens.

2
COLLECT INFORMATION

Logs, events, status, config.

3
ISOLATE THE CAUSE

Narrow down to component or layer.

4
IDENTIFY ROOT CAUSE

Find the exact reason.

5
APPLY SOLUTION

Make the fix and verify.

6
VALIDATE AND DOCUMENT

Confirm resolution and document it.

ESSENTIAL DOCKER COMPOSE COMMANDS

<code>docker compose ps</code>	- List running containers
<code>docker compose logs -f <svc></code>	- Follow logs for a service
<code>docker compose exec <svc> sh</code>	- Open shell in a container
<code>docker compose config</code>	- Validate and view final config
<code>docker compose top <svc></code>	- Show running processes
<code>docker compose port <svc> <port></code>	- Show published host port
<code>docker compose events</code>	- Stream real-time events
<code>docker compose down</code>	- Stop and remove containers
<code>docker compose down -v</code>	- Remove containers and volumes
<code>docker system df</code>	- Show disk usage

QUICK TIPS

- ✓ Always check logs first.
- ✓ Use service names, not localhost, between containers.
- ✓ Keep healthchecks accurate and meaningful.
- ✓ Validate config before deploying changes.
- ✓ Use named volumes for important data.
- ✓ Avoid unnecessary published ports.
- ✓ Reproduce in a minimal environment.



IMPORTANT

Never run containers as root. Secure your configuration, secrets, and network exposure.

PRODUCTION FAILURE SCENARIOS

COMMON ISSUES. REAL CAUSES. PROVEN SOLUTIONS.



1 DATABASE STARTS BUT IS NOT READY



- App tries to connect before DB is ready.
- Results in connection errors or crashes.

CHECK & FIX

- Add healthcheck to DB.
- Use `depends_on` with `condition: service_healthy`.
- Implement retry logic in the application.

2 APPLICATION ENTERS A RESTART LOOP



- App crashes immediately on start.
- Container restarts continuously.

CHECK & FIX

- Check container logs.
- Validate environment variables and configs.
- Fix code or configuration causing the crash.

3 CONTAINER RUNS OUT OF MEMORY



- Container is killed by OOM (Out Of Memory).
- Service becomes unavailable.

CHECK & FIX

- Check docker stats and logs.
- Increase memory limits.
- Optimize application memory usage.

4 VOLUME DATA DISAPPEARS



- Data missing after container restart or recreation.
- Volume not persisted or was removed.

CHECK & FIX

- Verify volume is named and mounted correctly.
- Avoid anonymous volumes.
- Check for accidental volume prune.

5 SERVICE NAME RESOLUTION FAILS



- Containers cannot resolve service names.
- Connection or DNS errors occur.

CHECK & FIX

- Ensure services are on same network.
- Check network config in `compose.yaml`.
- Restart Docker if needed.

6 NEW DEPLOYMENT BREAKS DEPENDENCIES



- New version incompatible with another service.
- Fails at runtime.

CHECK & FIX

- Review changelogs and version compatibility.
- Use semantic versioning and pin versions.
- Test in staging before production rollout.

7 CONFIGURATION DIFFERS BETWEEN ENVIRONMENTS



- App works in one env but fails in another.
- Different configs, secrets, or variables.

CHECK & FIX

- Compare environment files.
- Use `.env.example` and config validation.
- Keep configurations in version control.

8 ROOT-CAUSE INVESTIGATION FLOW



- Follow a structured process to find the true root cause.
- Avoid guesswork.

CHECK & FIX

- Follow the workflow below.
- Verify and document what you learn.

ROOT-CAUSE INVESTIGATION FLOW



DEFINE THE PROBLEM

- What is the issue?
- When does it occur?
- Who is affected?



COLLECT INFORMATION

- Check logs & events.
- Inspect containers.
- Review metrics.
- Check configs.



ISOLATE THE COMPONENT

- Identify the failing service or layer.
- Reproduce the issue in minimal setup.



IDENTIFY THE ROOT CAUSE

- Why is it failing?
- Check dependencies.
- Validate assumptions.



APPLY AND VERIFY FIX

- Implement the fix.
- Test the solution.
- Confirm the issue is resolved.



DOCUMENT AND PREVENT

- Document findings.
- Add monitoring.
- Prevent recurrence.

QUICK DIAGNOSTIC COMMANDS

<code>docker compose ps</code>	- Check container status
<code>docker compose logs <svc></code>	- View service logs
<code>docker compose exec <svc> sh</code>	- Open shell in a container
<code>docker stats</code>	- Live resource usage
<code>docker inspect <container></code>	- Inspect container details
<code>docker network inspect <net></code>	- Inspect network details
<code>docker volume ls</code>	- List volumes
<code>docker system df</code>	- Check disk usage
<code>docker events</code>	- Stream Docker events



PREVENTION BEST PRACTICES

- Use healthchecks on all critical services.
- Pin image versions and test updates.
- Validate configurations before deployment.
- Monitor logs, metrics, and alerts.
- Backup important data regularly.
- Test failure scenarios in staging.



REMEMBER

Most production issues are preventable.
Observability + Validation + Testing = Reliability.



COMPLETE PRODUCTION ARCHITECTURE

FINAL END-TO-END DOCKER COMPOSE ARCHITECTURE DIAGRAM

